简历工坊

作品介绍文档

传智杯 • AI Agent 智能助手 • 2026-10-30

一、项目详情及使用方式

简历工坊是一个「本地优先」的求职工作台：把「看懂岗位要求 → 判断匹配 → 定向修改简历 → 生成求职信 → 记录投递」串成一条可解释的链路。数据全部保存在访问者自己的浏览器里，服务端只承担 AI 代理，不存简历、不收集联系方式。

使用流程（约 3 分钟）：

① 打开首页 → 顶栏点「演示场景」，载入示例简历与 3 个目标岗位；

② 顶栏切到「AI 工作台」，左侧按匹配度列出待投岗位，选中一个；

③ 点「启动 AI 工作流」：模型先规划步骤，前两步本地执行（解析岗位、诊断差距），第三步流式改写简介并做事实核对，第四步生成求职信；

④ 点「采用并写入简介」，刷新后仍保留；不满意可一键撤销；

⑤ 投完点「标记为已投」，在投递看板里跟进。

运行方式：本地 `npm install && npm run dev`（默认 http://localhost:3000），或使用在线演示地址（部署在 Vercel，配置一个 DEEPSEEK_API_KEY 即可）。浏览器扩展可选：从 `extension/` 加载已解压扩展，可在招聘网站抓取真实 JD。

适用人群：应届生与 1–5 年经验的转岗者——他们有明确方向，但缺简历优化经验，也说不清自己与岗位差在哪。

二、项目亮点

1. 可解释的匹配，不是黑盒百分比

匹配分拆成「核心要求 55 + 加分技能 20 + 简历完整度 25」并逐项展示；核心要求按「同族技能任选其一」分组（React 与 Vue 不会同时被要求），缺口提示是「浏览器与性能优化」这种可执行的要求，而不是硬塞一个技能标签。

2. AI 是流程主导者，而不是右上角一个按钮

求职 Agent 先由模型规划步骤（工具白名单校验），再逐步执行；本地步骤跑完会把观察回灌给模型重新规划，用户也可以补充要求再规划。每一步都有状态、耗时与工具名，形成可查的工具调用轨迹。

3. 克制的 AI：能核对、可撤销、不替你编

生成后本地做事实核对（数字 / 技能 / 名称是否超出素材），结果明确展示但不阻断；必须点「采用」才写入简历，且写入进入撤销栈。没有 Key、断网或模型输出不合规时自动降级到本地计划。

4. 真实岗位数据，来自浏览器扩展

Chrome MV3 扩展从 BOSS 直聘 / 智联 / 猎聘等站点抓取当前岗位页的 JD，经本地消息桥进入待投清单。抓取采用 JSON-LD → 站点选择器 → 标题解析 → 正文兜底四层策略，并提供「诊断选择器」定位失效规则。只读当前页、不抓列表、不自动提交，规避平台条款与封号风险。

5. 隐私优先：数据不出浏览器

简历与投递记录存在 localStorage，AI 只发送当前板块文本、岗位关键词与参考 JD，不发送姓名、电话、邮箱与整份简历；API Key 只存在于服务端环境变量。

6. 交付完整：能打印、能导出 Word / PDF、能过 ATS、能离线

8 套模板、字体搭配、中英双语、手动分页与页码；双栏模板打印时改为「浮动侧栏 + 固定栏宽正文」，解决分栏无法跨页的问题。

导出菜单分两类：数据备份（全部 / 当前简历 JSON）与交付格式（Word `.docx`、PDF 图片版、ATS 文本 / Markdown）。Word 用单列结构、只含中文主字段，便于招聘系统解析；PDF 图片版一键下载、所见即所得；需要可选中文字的 PDF 时用「打印 / PDF」。PWA 外壳可离线打开。

7. 打开就能用：骨架屏 + 按需加载

首屏不再是白屏加一行「正在载入」，而是与真实布局同构的骨架（顶栏 / 选项卡 / 表单 / 纸张轮廓），服务端直接渲染。职位推荐、AI 工作台与两个导出弹窗按需加载，首屏只下载编辑页所需的代码。

Lighthouse 实测：性能 93、无障碍 100、最佳实践 96、SEO 100；本地生产构建在 4 倍 CPU 降速下首屏内容 0.76 秒出现。线上耗时主要来自跨境网络往返，已在文档中如实标注。

三、AI 技术落地使用情况

AI 在产品中承担三类任务，边界清晰：

环节	模型做什么	不做什么
单段写作	按岗位重写简介 / 经历 / 项目描述	不新增经历、不编造数字
整份生成	把一段自述整理成结构化简历	不生成姓名、联系方式
求职规划	决定接下来做哪几步、给出一句理由	不直接改简历、不调用未授权的工具

落地约束（工程上如何保证可用）：

- 输入裁剪：JD 2000 字、技能 20 个、观察 8 条封顶，避免无意义的长上下文与费用。
- 输出校验：规划结果必须是工具白名单内的 JSON，≤5 步、去重、超长截断；不合规直接丢弃。
- 失败降级：模型异常时重试一次，仍失败则回退本地规则计划；没有 Key 时整条链路仍可演示。
- 事实核对：每次生成后本地比对数字、技能、名称，列出待核对项。

质量数据（可复现，仓库内 `npm run eval` 生成）：

指标	结果
岗位画像识别	26 / 26
核心要求分组收益（逐项 82% → 分组 100%）	+18 个百分点
5 段手写 JD 的画像识别 / 缺口提示	5 / 5、5 / 5
事实核对：注入 20 条素材外事实	20 / 20 检出
事实核对：5 条干净文本	0 误报
单元测试 / 浏览器端到端	274 个用例、237 项检查

四、关键技术

技术	在项目中的作用
Next.js 16 App Router + React 19	页面静态渲染、AI 路由与页面同域（省去跨域与鉴权）
TypeScript strict	简历数据模型、匹配规则、Agent 步骤共享类型
SSE 流式输出	AI 文案逐字出现，长文本感知延迟显著下降
DeepSeek Chat + 结构化输出	短文本改写与规划，支持 JSON 模式与流式
提示词工程	角色约束、素材边界、语言标记 ===EN===、长度限制
本地存储与多标签页一致性	localStorage + storage 事件，未编辑页跟随、编辑页提示冲突
CSS @page 打印	分页控制、页码、双栏浮动降级
docx / jsPDF + html-to-image	浏览器端生成 Word 与图片版 PDF，无需服务端
Chrome MV3	抓取真实 JD、填充表单、选择器诊断
Vitest + Puppeteer	单元 / 组件测试 + 生产构建上的端到端回归

五、项目图片

以下截图均来自真实运行环境（演示场景 + 真实模型调用）。

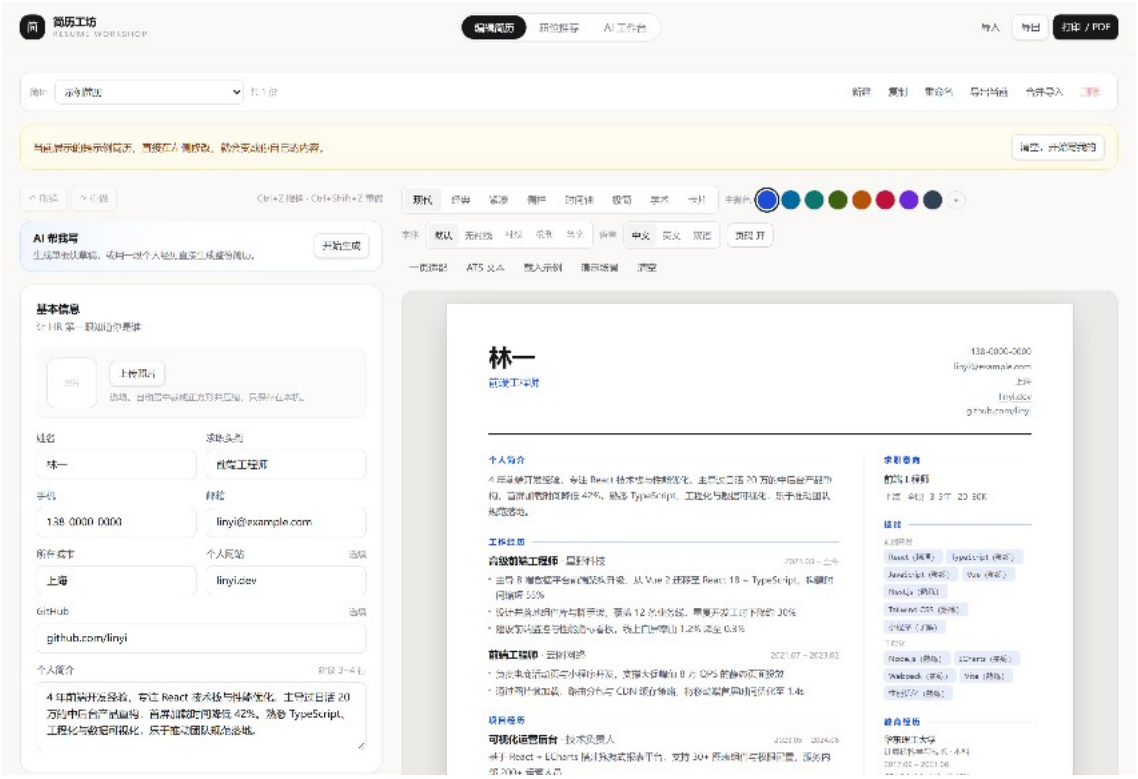


图 1　编辑页：左侧表单、右侧实时预览，可直接打印为 PDF

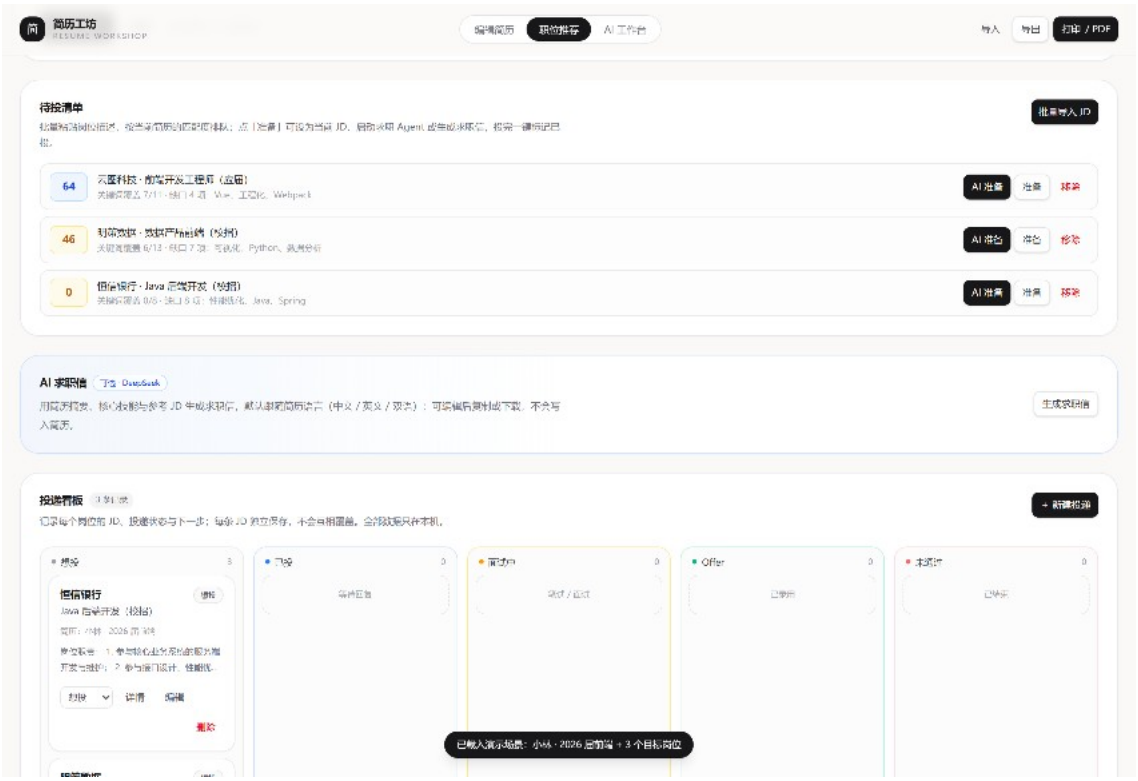


图 2　职位推荐：匹配分、核心要求命中与可点击的缺口



图 3　求职 Agent：四步计划、流式输出、事实核对与工具轨迹

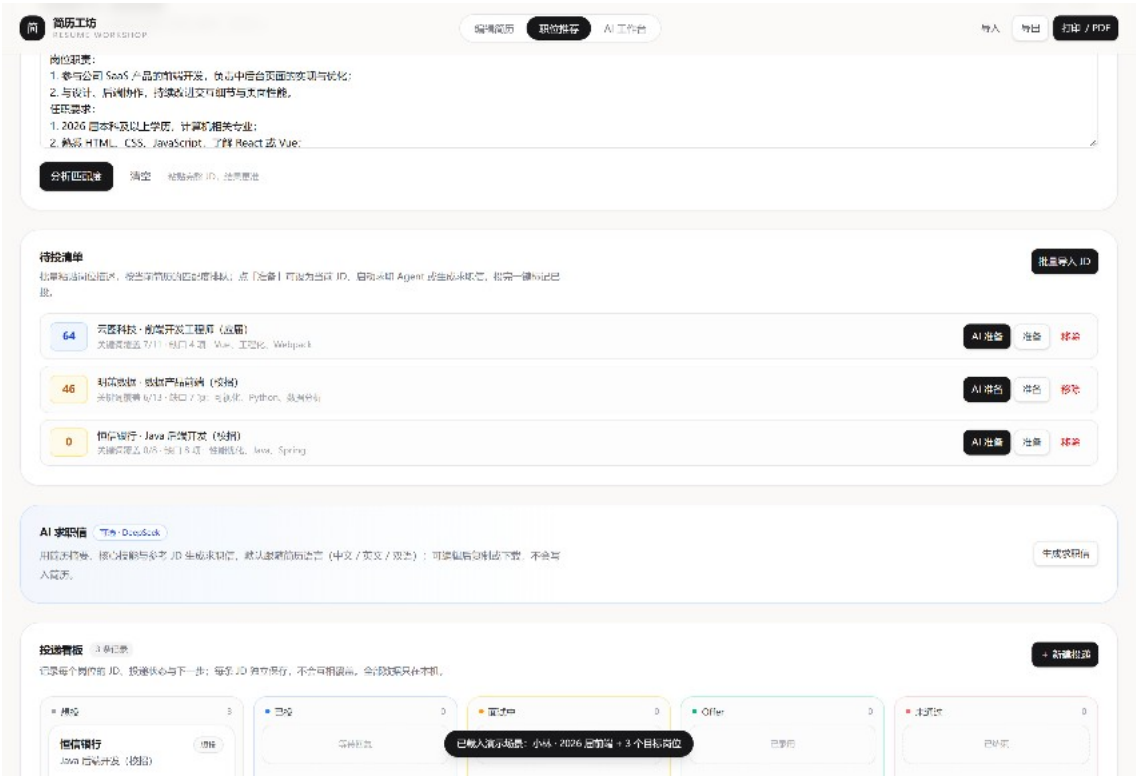


图 4　待投清单：批量导入的岗位按匹配度排序，右侧可直接「AI 准备」

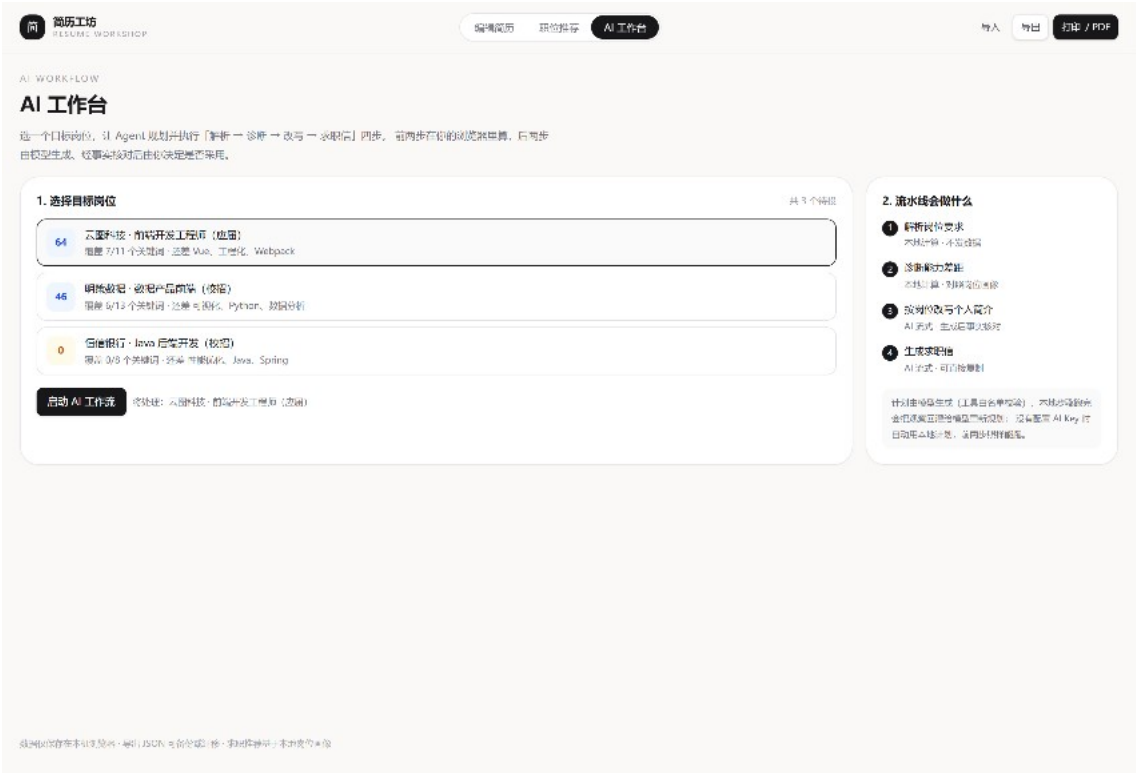


图 5　AI 工作台：左侧选目标岗位，右侧是四步流水线与执行说明

六、发表与获奖情况

本项目为本次参赛首次提交，此前未在任何平台发表、未参加其他赛事、未获得奖项。项目代码为独立开发，未使用任何开源项目模板或脚手架二次包装；所用依赖均为公开框架与库（Next.js、React、Tailwind CSS、Vitest、Puppeteer 等），在 `package.json` 中如实列出，前端无任何付费组件或素材。

七、原创性与参考说明

本项目不是在其他作品基础上的二次开发，代码与设计均为原创。开发过程中参考的公开资料限于：Next.js 官方文档（App Router、Route Handlers、打印相关 CSS）、DeepSeek 官方 API 文档（流式与 JSON 模式）、schema.org 的 JobPosting 规范（用于扩展解析招聘页的结构化数据）。简历模板与视觉设计为本项目自行实现，未使用第三方模板。

需要说明的是：产品定位与主流简历工具不同——它们聚焦「排版」，本项目聚焦「投什么、怎么投、投了没」，并通过浏览器扩展接入真实岗位数据、通过可解释的匹配与 Agent 工作流把 AI 放在流程主导位置。